WasmEdge LFX Mentorship (2026, Term 3) — Pre-test Report and Proposal

Support for the Wide Arithmetic Proposal (Issue #5153)

Basic Information

- Full Name: Ghule Vedant Babasaheb
- Email: vedant.ghule24@pccoepune.org
- GitHub: never954 — https://github.com/never954

Check List

- [x] I have read the WasmEdge Mentorship Program Policy (https://github.com/WasmEdge/WasmEdge/blob/master/docs/mentorship-policy.md)
- [x] I have read the LFX Mentorship Standards of Excellence (https://docs.linuxfoundation.org/lfx/mentorship/standards-of-excellence)

Pre-test 1 — Output of the C++ snippet

Output:

```
30
```

Pre-test 2 — SIMD instrumentation of the execution engine

Approach

In `Executor::execute` (`lib/executor/engine/engine.cpp`), the per-instruction dispatch reads the current opcode and runs a large `switch`. I inserted a check at the top of that dispatch, before the `switch`, that fires for every SIMD (v128) instruction and logs its mnemonic and byte offset.

WasmEdge generates its opcodes from one table (`include/common/enum.inc`), and every `0xFD`-prefixed SIMD opcode is emitted contiguously. Because the `OpCode` enum values follow that table order, all 256

SIMD opcodes occupy one unbroken numeric range, so a single range check identifies any SIMD instruction:

```
const OpCode Code = Instr.getOpCode();
if (Code >= OpCode::V128__load &&
    Code <= OpCode::I32x4__relaxed_dot_i8x16_i7x16_add_s) {
  spdlog::info("[SIMD] executing {} at offset {}"sv, Code, Instr.getOffset());
}
```

The mnemonic is printed through the existing `fmt::formatter<OpCode>`.

Patch (`simd-logging.patch`)

```
diff --git a/lib/executor/engine/engine.cpp b/lib/executor/engine/engine.cpp
--- a/lib/executor/engine/engine.cpp
+++ b/lib/executor/engine/engine.cpp
@@ -103,6 +103,12 @@ Expect<void> Executor::execute(Runtime::StackManager &StackMgr,
 #endif
   -> Expect<void> {
     const AST::Instruction &Instr = *PC;
+    const OpCode Code = Instr.getOpCode();
+    if (Code >= OpCode::V128__load &&
+        Code <= OpCode::I32x4__relaxed_dot_i8x16_i7x16_add_s) {
+      spdlog::info("[SIMD] executing {} at offset {}"sv, Code,
+                   Instr.getOffset());
+    }
     switch (Instr.getOpCode()) {
     // Control instructions
     case OpCode::Unreachable:
```

WASM file in WAT format (`simd.wat`)

```
(module
  (func (export "run") (result i32)
    v128.const i32x4 1 2 3 4
    v128.const i32x4 10 20 30 40
    i32x4.add
    i32x4.extract_lane 0))
```

Output (`wasmedge --reactor simd.wasm run`)

```
[info] [SIMD] executing v128.const at offset 33
[info] [SIMD] executing v128.const at offset 51
[info] [SIMD] executing i32x4.add at offset 69
[info] [SIMD] executing i32x4.extract_lane at offset 72
11
```

Pre-test 3 — LLVM i128 addition

Approach

A standalone program that uses the LLVM API to build a function `i128 add128(i128, i128)` with `IRBuilder`, JIT-compiles it with the `ExecutionEngine` (MCJIT), reads two 128-bit inputs, and prints the result.

Implementation (`llvm_i128.cpp`)

```
#include "llvm/ADT/APInt.h"
#include "llvm/ADT/SmallString.h"
#include "llvm/ExecutionEngine/ExecutionEngine.h"
#include "llvm/ExecutionEngine/MCJIT.h"
#include "llvm/IR/IRBuilder.h"
#include "llvm/IR/LLVMContext.h"
#include "llvm/IR/Module.h"
#include "llvm/IR/Verifier.h"
#include "llvm/Support/TargetSelect.h"
#include "llvm/Support/raw_ostream.h"
#include <cstdint>
#include <memory>
#include <string>

using namespace llvm;

static __int128 apintToI128(const APInt &V) {
  uint64_t Lo = V.getRawData()[0];
  uint64_t Hi = V.getRawData()[1];
  return (__int128)(((unsigned __int128)Hi << 64) | Lo);
}
static APInt i128ToAPInt(__int128 X) {
  unsigned __int128 U = (unsigned __int128)X;
  uint64_t Words[2] = {(uint64_t)U, (uint64_t)(U >> 64)};
  return APInt(128, ArrayRef<uint64_t>(Words, 2));
}

int main(int argc, char **argv) {
  StringRef AStr = (argc > 1) ? argv[1] : "170141183460469231731687303715884105727";
  StringRef BStr = (argc > 2) ? argv[2] : "1";
  APInt A(128, AStr, 10), B(128, BStr, 10);

  InitializeNativeTarget();
  InitializeNativeTargetAsmPrinter();

  auto Ctx = std::make_unique<LLVMContext>();
  auto Mod = std::make_unique<Module>("wide_arith", *Ctx);
  IRBuilder<> Builder(*Ctx);

  Type *I128 = Builder.getInt128Ty();
  FunctionType *FT = FunctionType::get(I128, {I128, I128}, false);
  Function *F = Function::Create(FT, Function::ExternalLinkage, "add128", Mod.get());

  BasicBlock *Entry = BasicBlock::Create(*Ctx, "entry", F);
  Builder.SetInsertPoint(Entry);
  auto Args = F->arg_begin();
  Value *ArgA = &*Args++;
  Value *ArgB = &*Args;
```

```
  ArgA->setName("a"); ArgB->setName("b");
  Value *Sum = Builder.CreateAdd(ArgA, ArgB, "sum");
  Builder.CreateRet(Sum);

  if (verifyFunction(*F, &errs())) { errs() << "verify failed\n"; return 1; }
  outs() << "=== Generated LLVM IR ===\n";
  Mod->print(outs(), nullptr);

  std::string ErrStr;
  ExecutionEngine *EE = EngineBuilder(std::move(Mod))
                            .setErrorStr(&ErrStr)
                            .setEngineKind(EngineKind::JIT)
                            .create();
  if (!EE) { errs() << "EE: " << ErrStr << "\n"; return 1; }
  EE->finalizeObject();

  auto Add128 = (__int128 (*)(__int128, __int128))EE->getFunctionAddress("add128");
  __int128 Result = Add128(apintToI128(A), apintToI128(B));

  SmallString<64> AS, BS, RS;
  A.toString(AS, 10, false);
  B.toString(BS, 10, false);
  i128ToAPInt(Result).toString(RS, 10, false);
  outs() << "=== Result ===\n" << AS << " + " << BS << " = " << RS << "\n";
  delete EE;
  return 0;
}
```

Build command

```
clang++ $(llvm-config --cxxflags) llvm_i128.cpp \
  $(llvm-config --ldflags --libs core executionengine mcjit native --system-libs) \
  -o llvm_i128
```

Output

Default inputs (`INT128_MAX + 1`):

```
=== Generated LLVM IR ===
define i128 @add128(i128 %a, i128 %b) {
entry:
  %sum = add i128 %a, %b
  ret i128 %sum
}
=== Result ===
170141183460469231731687303715884105727 + 1 = 170141183460469231731687303715884105728
```

Proposal

Summary

This project implements the four WebAssembly Wide Arithmetic instructions — `i64.add128`, `i64.sub128`, `i64.mul_wide_s`, `i64.mul_wide_u` — across all of WasmEdge's execution modes: the interpreter, the LLVM AOT compiler, and the JIT engine. The work spans opcode registration and proposal gating, binary loader parse/serialize, validator stack typing, interpreter execution, and LLVM lowering, verified against the upstream specification tests.

I have built and run a working interpreter prototype for all four instructions against a local `master` checkout (see Prototype, below).

Problem statement

WebAssembly natively supports integers only up to 64 bits. Programs needing 128-bit arithmetic — cryptography, big-integer libraries, blockchain — emulate it with sequences of 64-bit operations plus explicit carry/borrow detection (a wrap-around comparison and branch after each partial add), turning one 128-bit add into several instructions. The hardware already does this efficiently through the carry flag and add-with-carry; core WebAssembly simply had no way to express it. The Wide Arithmetic proposal adds that vocabulary so a runtime can lower these operations to native carry/borrow and widening-multiply instructions — smaller binaries and materially faster big-integer code, most notably through WasmEdge's LLVM AOT/JIT path where `i64.add128` becomes a native `add`/`adc` pair.

Instruction semantics

Instruction	Opcode	Stack type	Meaning
`i64.add128`	`0xFC 19`	`[i64 i64 i64 i64] -> [i64 i64]`	128-bit add of two (lo, hi) pairs
`i64.sub128`	`0xFC 20`	`[i64 i64 i64 i64] -> [i64 i64]`	128-bit subtract (LHS – RHS)
`i64.mul_wide_s`	`0xFC 21`	`[i64 i64] -> [i64 i64]`	signed 64×64 → 128-bit product
`i64.mul_wide_u`	`0xFC 22`	`[i64 i64] -> [i64 i64]`	unsigned 64×64 → 128-bit product

Operand order from the spec: for `add128`/`sub128`, the stack top-to-bottom is `RHS_high, RHS_low, LHS_high, LHS_low`; `v1 = LHS`, `v2 = RHS`, and `sub128` computes `v1 - v2`. The `0xFC` opcode block in WasmEdge currently ends at sub-opcode 17 (`table.fill`), so 19–22 are free.

Implementation architecture

Verified against current `master`.

1. Opcode registration and proposal gating

`include/common/enum.inc` — add four opcode entries for sub-opcodes 19–22 and a new `WideArithmetic` proposal, gating the opcodes behind it. The loader's decoder is generated from this same table, so registration makes the opcodes decodable automatically. WasmEdge enables proposals by default and exposes `--disable-<proposal>` toggles, so gating means adding `WideArithmetic` to the default set, adding a disable toggle, and adding `hasProposal()` checks.

2. Loader — parse and serialize

`lib/loader/ast/instruction.cpp` and `lib/loader/serialize/serial_instruction.cpp`. The four instructions carry no immediate operands, so they join the no-immediate group on the parse side and are mirrored on the serialize side, with loader round-trip tests.

3. Validator — stack typing

`lib/validator/formchecker.cpp`. Stack effects are expressed through a generic `StackTrans(Take, Put)` (`popTypes(Take); pushTypes(Put)`). Because the Multi-value proposal is already supported, two-result instructions need no new machinery — only the correct signatures:

- `add128` / `sub128` — `StackTrans({I64,I64,I64,I64}, {I64,I64})`
- `mul_wide_*` — `StackTrans({I64,I64}, {I64,I64})`

4. Interpreter — execution

`lib/executor/engine/engine.cpp`, beside the existing `I64__add` case. The 128-bit math uses `WasmEdge::uint128_t` / `int128_t` from `include/common/int128.h`, which is native `__int128` on GCC/Clang and a struct-with-operators fallback on MSVC, so the Windows build stays correct without special-case intrinsics. Reconstructing the full 128-bit operands and using a single `+` / `-` / `*` lets the host compiler emit the native `add` / `adc` (or borrow / widening-multiply) sequence, replacing the emulated compare-and-branch carry detection.

5. LLVM AOT/JIT — lowering

`lib/llvm/compiler/numericInstr.cpp`, beside the `I64__add` lowering (`Builder.createAdd`). For `add128` / `sub128`: zero-extend the four `i64`s to `i128`, reassemble the operands, `createAdd` / `createSub`, then split the result into two `i64`s (`trunc` low; `lshr 64` + `trunc` high). For `mul_wide_u` / `mul_wide_s`: `zext` / `sext` both `i64` to `i128`, `createMul`, split. The AOT stack is a plain vector, so pushing two results is two `stackPush` calls. LLVM folds this IR into the native carry / widening-multiply instructions.

Prototype: a working interpreter path

I implemented the complete interpreter path for all four instructions in a local build of `master`. The change is a 61-line diff across four files — stages 1–4 above:

```
include/common/enum.inc        |  6 ++++++
lib/executor/engine/engine.cpp | 40 ++++++++++++++++++++++++++++++++++++++++
lib/loader/ast/instruction.cpp |  6 ++++++
lib/validator/formchecker.cpp  |  9 +++++++++
```

It parses, validates, and executes a `.wat` module using all four instructions. Results printed as signed `i64`:

Function	Inputs	Result (lo, hi)
`add128`	$(2^{64} - 1) + 1$	`0, 1`
`sub128`	$2^{64} - 1$	`-1, 0`
`mul_wide_u`	$(2^{64} - 1)^2$	`1, -2`
`mul_wide_s`	$(-1) \cdot (-1)$	`1, 0`

The prototype exercises the non-commutative `sub128 = LHS − RHS` and establishes the test vectors reused for the AOT/JIT differential tests.

Testing strategy

- Differential tests across interpreter, AOT, and JIT: zero, carry across the 64-bit boundary, signed vs unsigned multiply, `INT64_MIN`/max edges.
- Upstream spec tests: the suite is fetched from `WasmEdge/wasmedge-spectest` (currently pinned at `ee503c07`, `wasm-core-20260728`) via `test/spec/CMakeLists.txt`. `WASMEDGE_SPECTEST_FOLDERS` does not yet list a `wide-arithmetic` entry, and the upstream `WebAssembly/wide-arithmetic` repo ships `test/core/wide-arithmetic.wast` covering all four instructions. Making the spec run green therefore requires those tests in the WasmEdge spectest mirror with the pin bumped. Open question for the mentors in Week 1: should the mentee PR the `wide-arithmetic` folder into `wasmedge-spectest` and bump the tag, or is that mirror updated on a separate maintainer cadence? This determines whether spec integration is my task or a dependency, and it is scheduled explicitly.
- Loader round-trip tests under `test/loader` for the new opcodes.

12-week timeline

Weeks	Milestone
1	Pin operand order to the spec; confirm spec-test mirror ownership; add `WideArithmetic` proposal flag and four opcodes; wire configuration gating.
2	Loader parse and serialize; loader/serializer round-trip tests.
3	Validator signatures; validation unit tests (accept and reject).
4	Finalize interpreter handlers (building on the prototype) and edge-case tests.
5	Interpreter hardening; buffer.
6–7	LLVM lowering for `add128` / `sub128`; correct on both AOT and JIT.
8–9	LLVM lowering for `mul_wide_s` / `mul_wide_u`; differential tests across all three engines.
10	Spec-test integration per the Week 1 decision; drive to green.
11	Cross-platform hardening (MSVC path via `int128.h`); review iteration; buffer.
12	Documentation in `WasmEdge/docs`; final review and feedback.

Risks and mitigations

- Spec-test mirror dependency — surfaced in Week 1 so it never blocks Week 10.
- Operand-order and lo–hi errors — pin to the spec first and rely on differential testing across engines (the prototype already exercises this, including the non-commutative `sub128`).
- LLVM API drift — build against the LLVM version CI uses; keep lowering to stable `IRBuilder` primitives (`zext`, `sext`, `shl`, `lshr`, `trunc`, `add`, `sub`, `mul`).

Availability

I have university mid-term and end-semester examinations from late November to early December, lasting roughly 15–20 days, during which my weekly progress will be lower than usual. The timeline front-loads the interpreter and validator work and keeps buffer weeks, so this period lands on lighter integration and documentation tasks rather than the core implementation. I will share the exact dates with the mentors at the start and adjust the affected weeks accordingly.